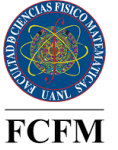




UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE CIENCIAS FISICO-MATEMATICAS



Laboratorio Programación Orientada a Objetos

Práctica #11 - Factory Singleton (Threads)

Profesor: Jorge Alberto Islas Pineda

Alumno: Jorge Jared Ramírez González

Matrícula: 2055454

Grupo: 032

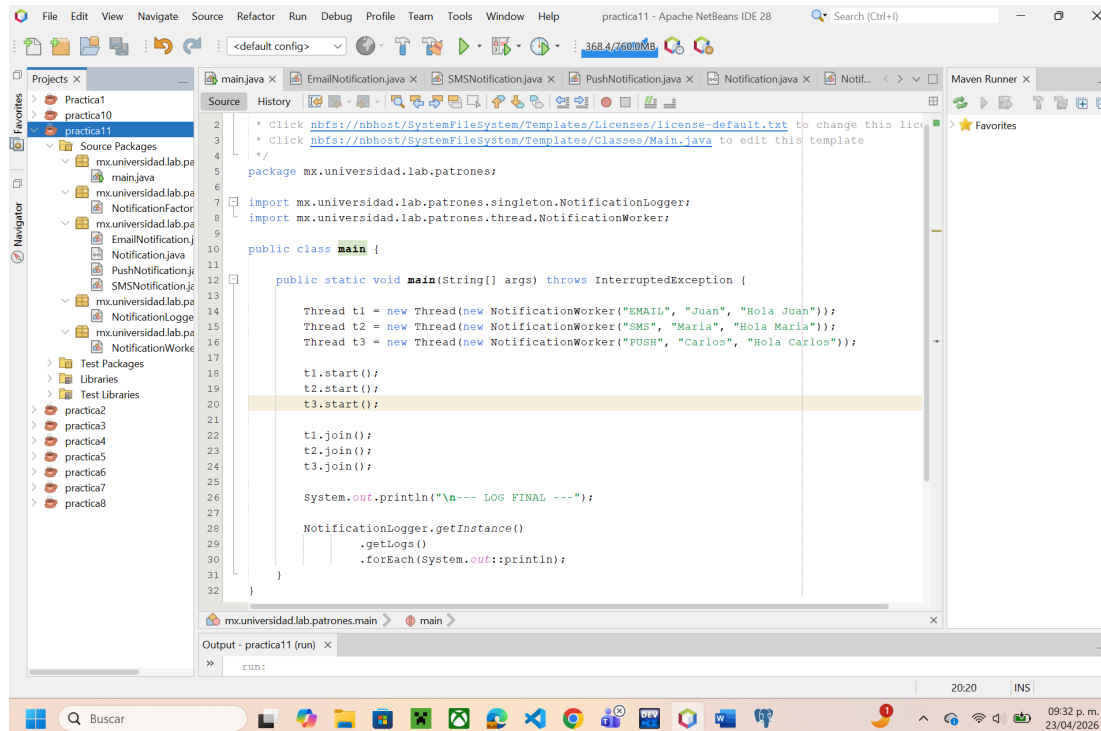
Ciudad Universitaria 23 de abril 2026

Práctica #11:Factory Singleton (Threads)

Código.

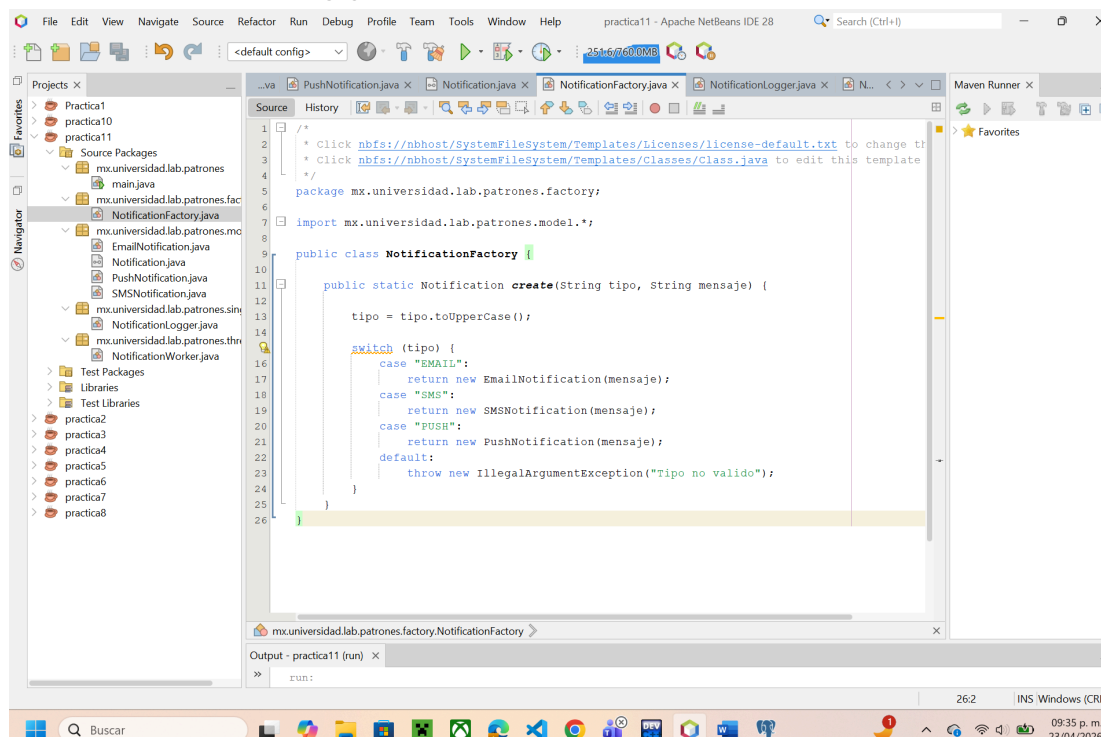
package mx.universidad.lab.patrones

main.java

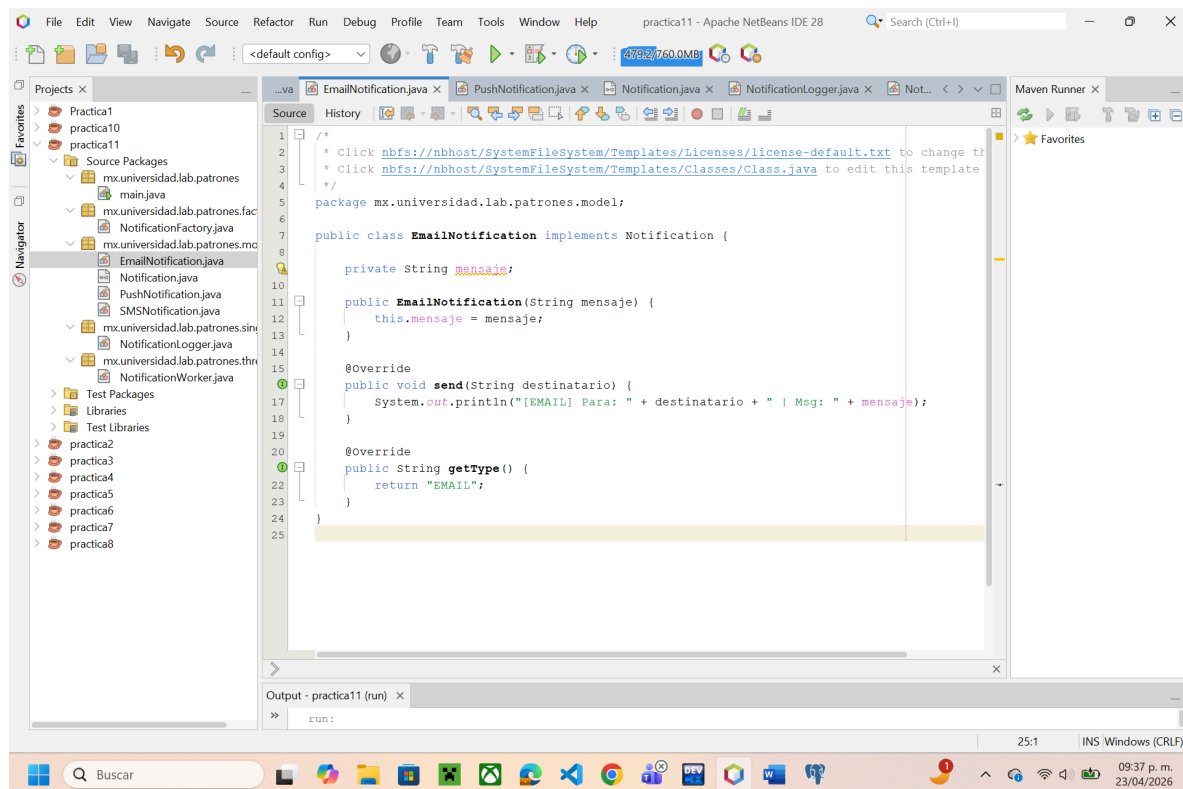


package mx.universidad.lab.patrones.factory

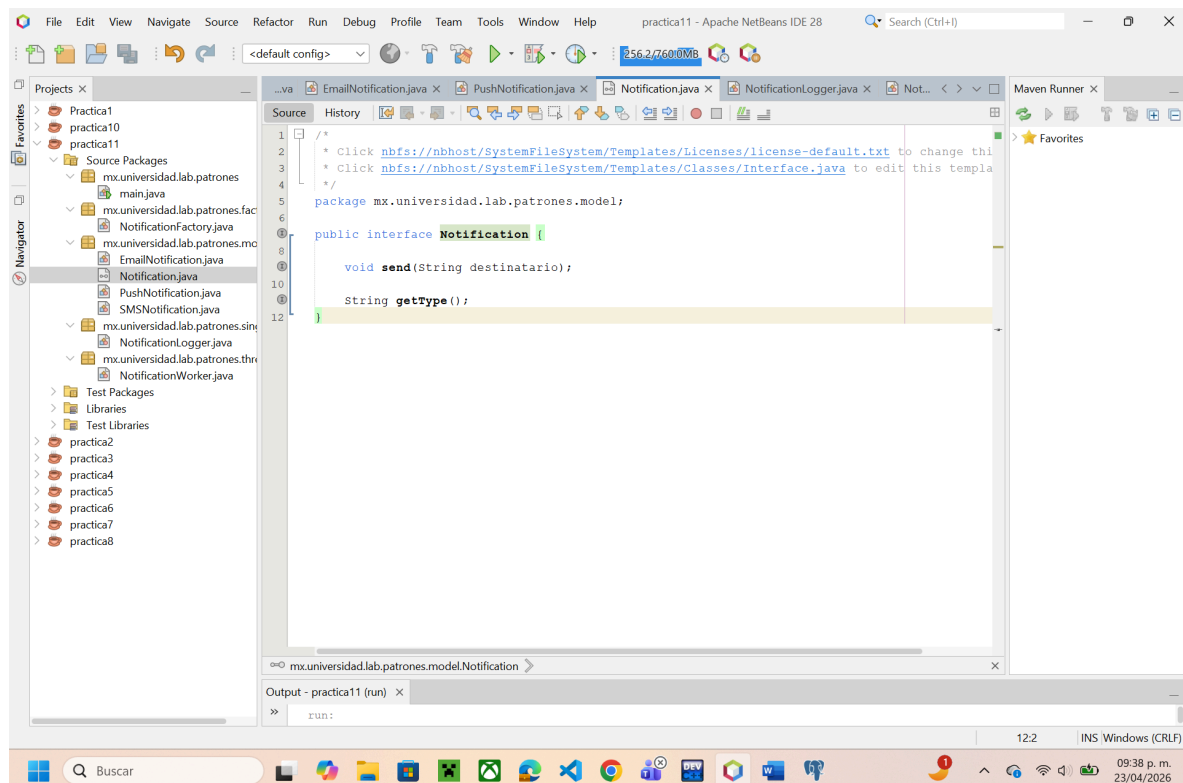
NotificationFactory.java



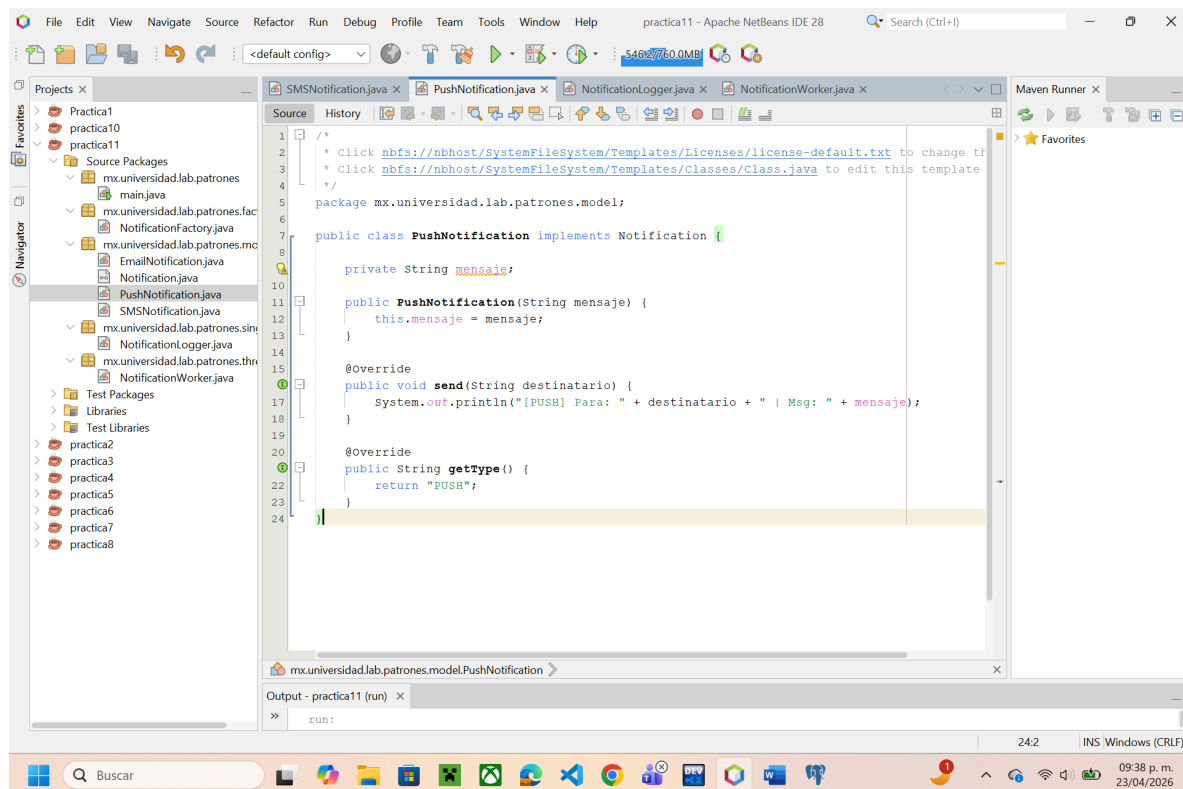
package mx.universidad.lab.patrones.model EmailNotification.java



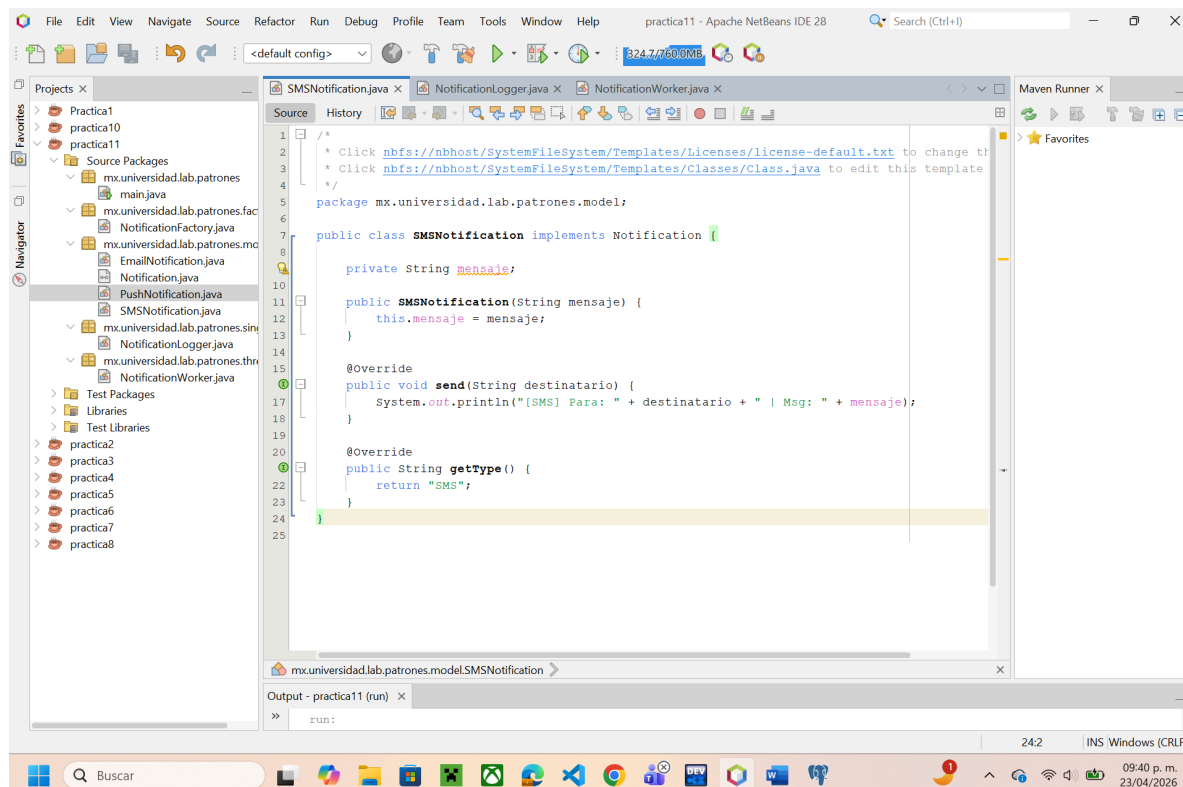
package mx.universidad.lab.patrones.model Notification.java



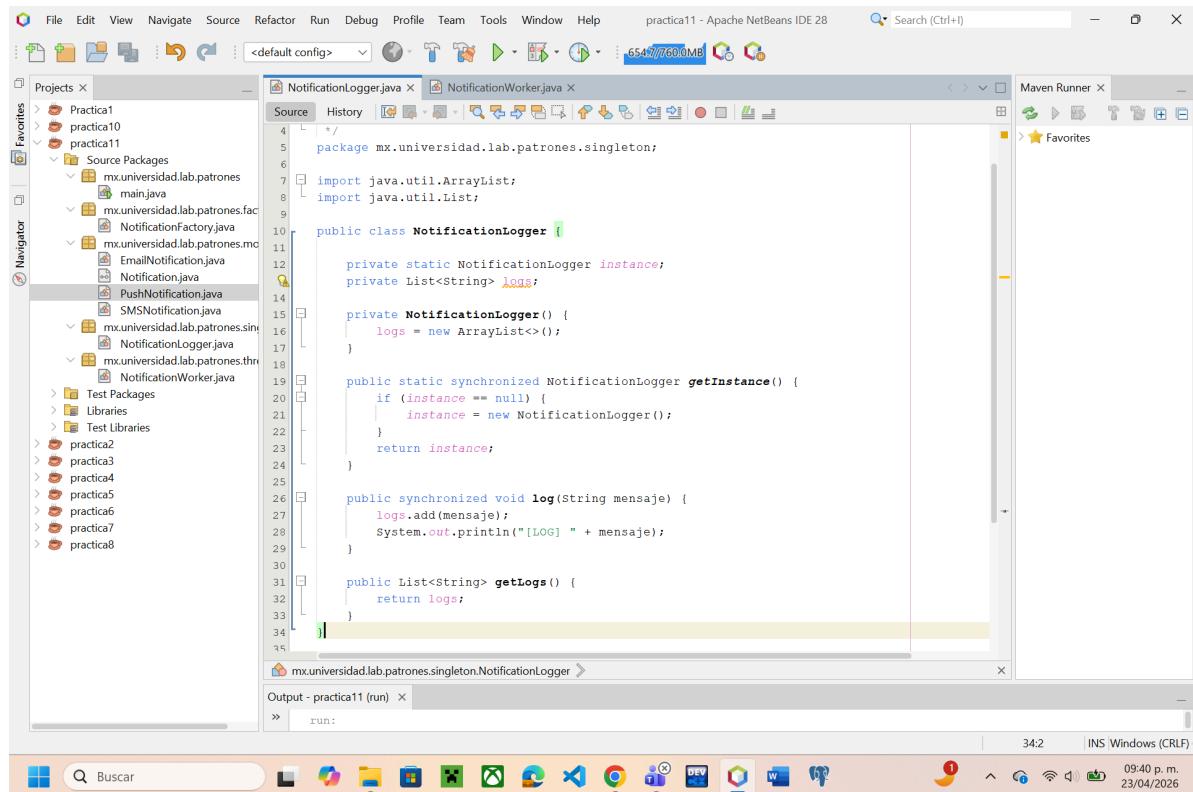
package mx.universidad.lab.patrones.model PushNotification.java



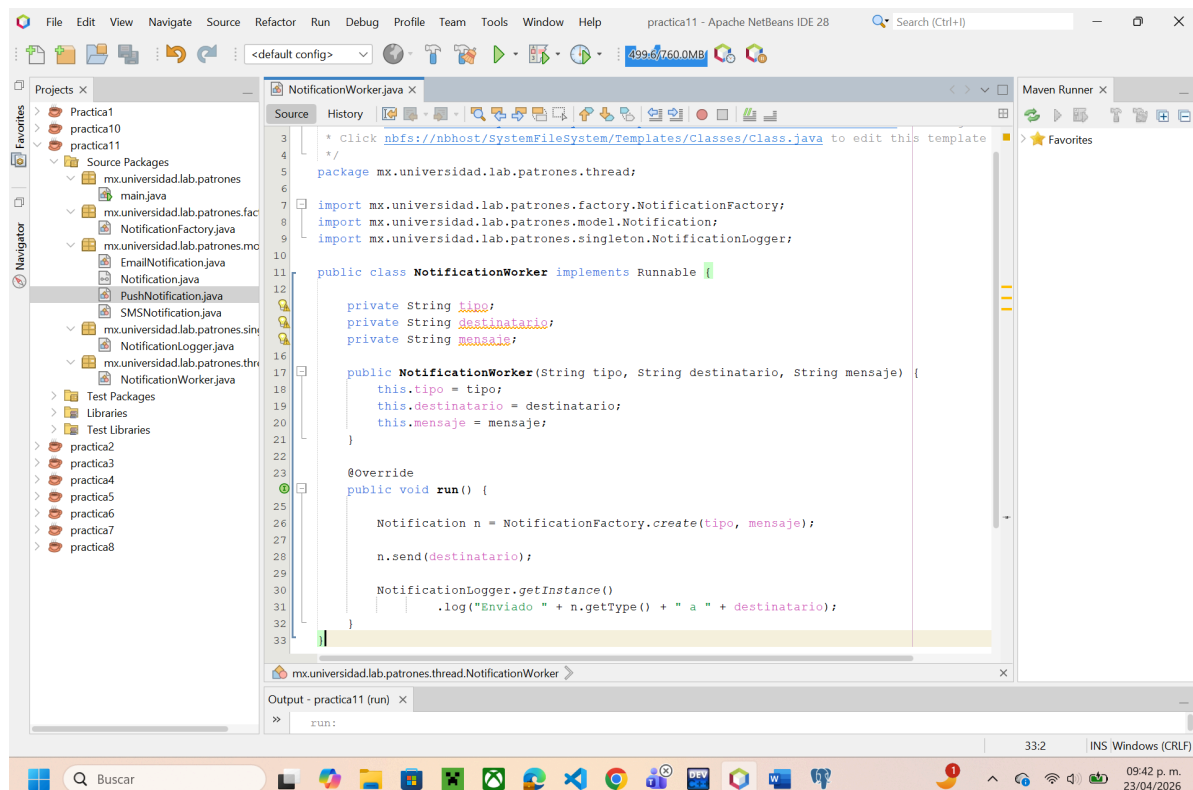
package mx.universidad.lab.patrones.model SMSNotification.java



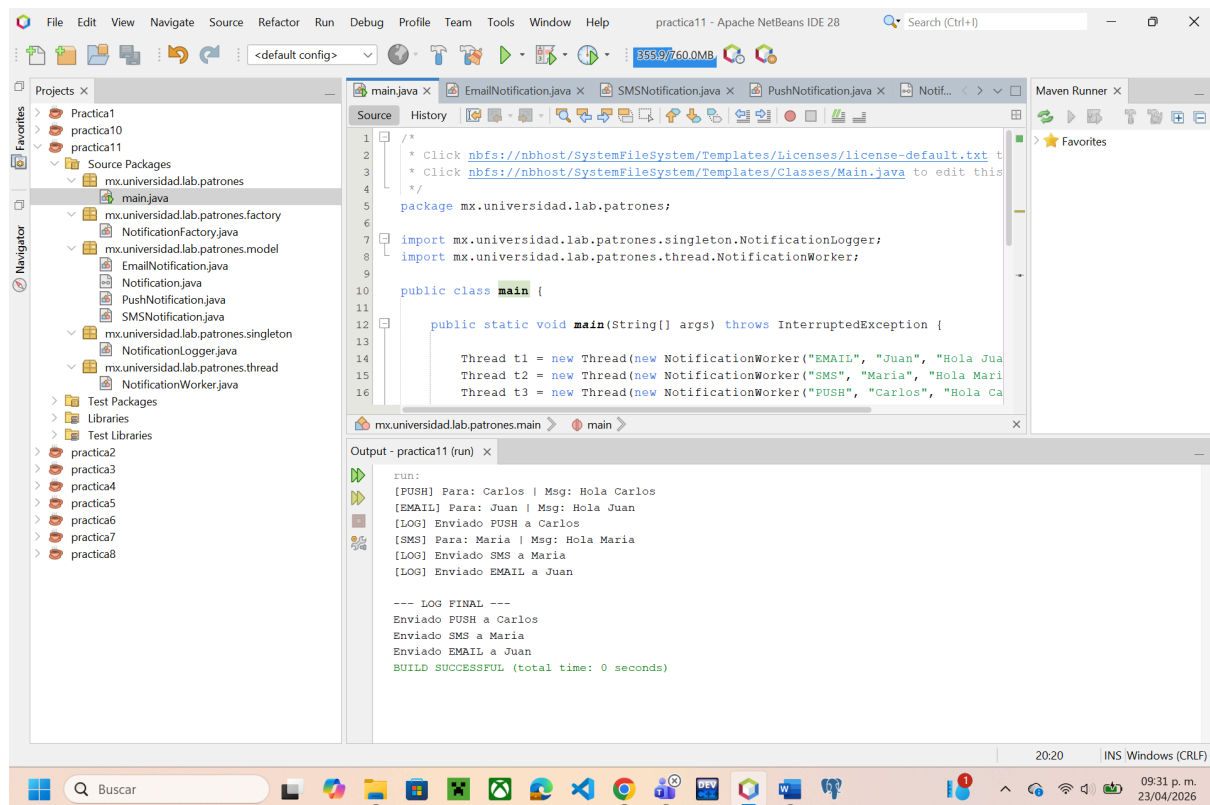
package mx.universidad.lab.patrones.singleton NotificationLogger.java



package mx.universidad.lab.patrones.thread NotificationWorker.java



Output.



```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/licenses/license-default.txt to
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this
4  */
5  package mx.universidad.lab.patrones;
6
7  import mx.universidad.lab.patrones.singleton.NotificationLogger;
8  import mx.universidad.lab.patrones.thread.NotificationWorker;
9
10 public class main {
11
12     public static void main(String[] args) throws InterruptedException {
13
14         Thread t1 = new Thread(new NotificationWorker("EMAIL", "Juan", "Hola Jua
15         Thread t2 = new Thread(new NotificationWorker("SMS", "Maria", "Hola Mari
16         Thread t3 = new Thread(new NotificationWorker("PUSH", "Carlos", "Hola Ca
```

```
run:
[PUSH] Para: Carlos | Msg: Hola Carlos
[EMAIL] Para: Juan | Msg: Hola Juan
[LOG] Enviado PUSH a Carlos
[LOG] Enviado SMS a Carlos
[LOG] Enviado EMAIL a Carlos
[LOG] Enviado PUSH a Maria
[LOG] Enviado SMS a Maria
[LOG] Enviado EMAIL a Maria
[LOG] Enviado PUSH a Juan
[LOG] Enviado SMS a Juan
[LOG] Enviado EMAIL a Juan

--- LOG FINAL ---
Enviado PUSH a Carlos
Enviado SMS a Maria
Enviado EMAIL a Juan
BUILD SUCCESSFUL (total time: 0 seconds)
```

Reflexión.

En esta práctica se aplicaron tres conceptos importantes de la programación: el patrón Factory, el patrón Singleton y el uso de hilos (Threads).

El patrón Factory permite crear objetos sin necesidad de especificar directamente la clase concreta, lo que hace el código más flexible y fácil de mantener. En este caso, se utilizó para crear distintos tipos de notificaciones como Email, SMS y Push.

El patrón Singleton se utilizó en la clase NotificationLogger para asegurar que solo exista una única instancia del logger en todo el sistema. Esto es importante porque permite centralizar el registro de mensajes y evitar inconsistencias en la información.

El uso de synchronized en el Singleton es necesario debido a que múltiples hilos pueden acceder al logger al mismo tiempo. Sin esta sincronización, podrían ocurrir errores como datos incorrectos o pérdida de información.

Finalmente, el uso de Threads permite ejecutar varias tareas de forma concurrente, simulando el envío de múltiples notificaciones al mismo tiempo, lo cual representa un escenario real en sistemas modernos. En conclusión, esta práctica permitió integrar conceptos avanzados de programación orientada a objetos y concurrencia para construir un sistema más eficiente y organizado.